

programming languages are available at <http://bit.ly/rItE>.

It is interesting to note that, by 2013, the scene is much more democratic, with many languages being quite popular—the language at first position (namely C) has only 17.85 per cent of the overall votes, with Java being a close second with 17.41 per cent. This is a major shift in trend from 2003, when C, C++, and Java accounted for nearly 70 per cent of the overall popularity votes. We consider this an important trend, which is likely to continue.

This is significant for programmers, because it is no longer enough for you to be an expert in a single programming language, say C or C++ or Java. Today, the expectations are that given a particular software project, you should pick up the language that is most suited for it, rather than sticking with the language of your expertise. Therefore, programmers are expected to be experts in multiple languages, and keep their skills sharpened as newer programming languages appear on the scene. For example, consider Scala, which appeared on the scene in 2003 as a better form of Java, bringing functional programming to Java. Today, Scala is employed by major deployments such as the Twitter platform, appearing as the 33rd most popular programming language on the TIOBE index. On the alternate popular programming language index <http://redmonk.com/sogradi/2012/09/12/language-rankings-9-12/> Scala appears in the 12th position (RedMonk ratings are based on usage information from <http://www.stackoverflow.com> and GitHub). A number of new programming languages have appeared on the scene. These include ‘GO’ and ‘Lua’, and are gaining popularity.

One of the other interesting facts about the evolution of programming languages is the type of language features that have appeared in them. Functional programming support, dynamic typing and concurrency support are some of the interesting features to look at. Almost every imperative language—C, C++ or Java—has been trying to borrow elements from functional programming. Language support for Lambda expressions is today available in C++ and Java directly, and a plethora of libraries exist to complement functional language support for Java and C++. While Java has been supporting concurrency as a first-class citizen from the beginning, C++ recently added concurrency support in the language. While statically typed languages like C, C++ and Java still continue to rule the roost, many popular scripting languages like Python, JavaScript and Ruby are dynamically typed. Another point to note is the rise of scripting languages support for JIT compilation with Python—Ruby and JavaScript support JIT compilation in various forms.

The road ahead for the next 10 years

We have seen languages moving from being closer to the hardware, such as C and assembly language, to higher-level

Table 1

Position	Programming Language
1	Java
2	C
3	C++
4	Perl
5	Visual Basic
6	PHP
7	SQL
8	C#
9	JavaScript
10	Delphi/Pascal

Table 2

Position	Programming Language
1	C
2	Java
3	Objective-C
4	C++
5	C#
6	PHP
7	Visual Basic
8	Python
9	Perl
10	JavaScript

languages that run on virtual machines, with the underlying hardware being abstracted to a virtual machine for the language writer. Portability and programmer productivity have become the dominant themes, instead of squeezing out the last drop of performance by writing assembly language code. This trend is likely to continue further over the coming years.

Given the multi-core evolution in computer architecture, effective support for concurrency will be another dominant theme for programming languages. Unlike imperative languages like C, C++ and Java, which support the notion of program state through shared variables, functional programming is all about functions as first-class citizens, and more importantly, the absence of side-effects such as global variable updates. Therefore, threading and parallelism become quite straightforward in functional languages. We are likely to see increasing support for functional programming being added to imperative languages. Given the difficulties associated with performance-efficient concurrent programming in languages like C, C++ and Java, support for transactional memory is another trend likely to continue in programming languages.

I believe that a far more significant change that will take place over the next 10 years is the democratisation of programming. Programming software systems will not be limited to a select few expert programmers, but the languages will evolve and become so much simpler that most people will be able to write programs. The stereotypical geek programmer will slowly give way to the lay-person programmer who has domain expertise. It is not necessary to have advanced, in-depth knowledge in programming to write working code in Ruby or Python. The advent of libraries and frameworks makes it easy to write code quickly with minimal effort. Instead of writing software systems from scratch, programming will be more about writing glue code that ties together functionality from different libraries and frameworks. This trend will be driven predominantly by open source software.